Boosting

What is boosting in machine learning?

Boosting is a method used in machine learning to reduce errors in predictive data analysis. Data scientists train machine learning software, called machine learning models, on labeled data to make guesses about unlabeled data. A single machine learning model might make prediction errors depending on the accuracy of the training dataset. For example, if a cat-identifying model has been trained only on images of white cats, it may occasionally misidentify a black cat. Boosting tries to overcome this issue by training multiple models sequentially to improve the accuracy of the overall system.

Why is boosting important?

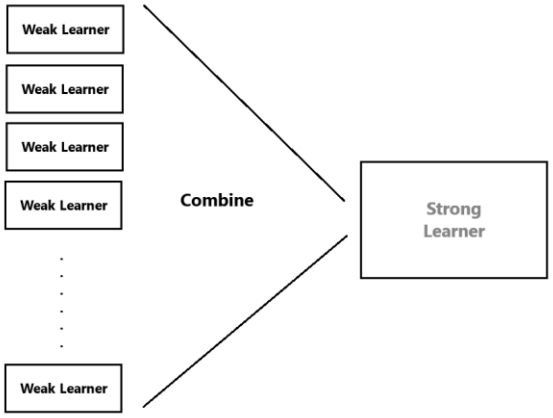
Boosting improves machine models' predictive accuracy and performance by converting multiple weak learners into a single strong learning model. Machine learning models can be weak learners or strong learners:

Weak learners

Weak learners have low prediction accuracy, similar to random guessing. They are prone to overfitting—that is, they can't classify data that varies too much from their original dataset. For example, if you train the model to identify cats as animals with pointed ears, it might fail to recognize a cat whose ears are curled.

Strong learners

Strong learners have higher prediction accuracy. Boosting converts a system of weak learners into a single strong learning system. For example, to identify the cat image, it combines a weak learner that guesses for pointy ears and another learner that guesses for cat-shaped eyes. After analyzing the animal image for pointy ears, the system analyzes it once again for cat-shaped eyes. This improves the system's overall accuracy.



How does boosting work?

To understand how boosting works, let's describe how machine learning models make decisions. Although there are many variations in implementation, data scientists often use boosting with decision-tree algorithms:

Decision trees

Decision trees are data structures in machine learning that work by dividing the dataset into smaller and smaller subsets based on their features. The idea is that decision trees split up the data repeatedly until there is only one class left. For example, the tree may ask a series of yes or no questions and divide the data into categories at every step.

Boosting ensemble method

Boosting creates an ensemble model by combining several weak decision trees sequentially. It assigns weights to the output of individual trees. Then it gives incorrect classifications from the first decision tree a higher weight and input to the next tree. After numerous cycles, the boosting method combines these weak rules into a single powerful prediction rule.

Boosting compared to bagging

Boosting and bagging are the two common ensemble methods that improve prediction accuracy. The main difference between these learning methods is the method of training. In bagging, data scientists improve the accuracy of weak learners by training several of them at once on multiple datasets. In contrast, boosting trains weak learners one after another.

How is training in boosting done?

The training method varies depending on the type of boosting process called the boosting algorithm. However, an algorithm takes the following general steps to train the boosting model:

Step 1

The boosting algorithm assigns equal weight to each data sample. It feeds the data to the first machine model, called the base algorithm. The base algorithm makes predictions for each data sample.

Step 2

The boosting algorithm assesses model predictions and increases the weight of samples with a more significant error. It also assigns a weight based on model performance. A model that outputs excellent predictions will have a high amount of influence over the final decision.

Step 3

The algorithm passes the weighted data to the next decision tree.

Step 4

The algorithm repeats steps 2 and 3 until instances of training errors are below a certain threshold.

What are the types of boosting?

1. AdaBoost
2. Gradient Boosting machine

3. Extreme Gradient Boost
4. Light Gradient Boost Machine
5. catBoost

Adaptive boosting

Adaptive Boosting (AdaBoost) was one of the earliest boosting models developed. It adapts and tries to self-correct in every iteration of the boosting process.

AdaBoost initially gives the same weight to each dataset. Then, it automatically adjusts the weights of the data points after every decision tree. It gives more weight to incorrectly classified items to correct them for the next round. It repeats the process until the residual error, or the difference between actual and predicted values, falls below an acceptable threshold.

You can use AdaBoost with many predictors, and it is typically not as sensitive as other boosting algorithms. This approach does not work well when there is a correlation among features or high data dimensionality. Overall, AdaBoost is a suitable type of boosting for classification problems.

Gradient boosting

Gradient Boosting (GB) is similar to AdaBoost in that it, too, is a sequential training technique. The difference between AdaBoost and GB is that GB does not give incorrectly classified items more weight. Instead, GB software optimizes the loss function by generating base learners sequentially so that the present base learner is always more effective than the previous one. This method attempts to generate accurate results initially instead of correcting errors throughout the process, like AdaBoost. For this reason, GB software can lead to more accurate results. Gradient Boosting can help with both classification and regression-based problems.

Extreme gradient boosting

Extreme Gradient Boosting (XGBoost) improves gradient boosting for computational speed and scale in several ways. XGBoost uses multiple cores on the CPU so that learning can occur in parallel during training. It is a boosting algorithm that can handle extensive datasets, making it attractive for big data applications. The key features of XGBoost are parallelization, distributed computing, cache optimization, and out-of-core processing.

LightGBM (Light Gradient Boosting Machine)

LightGBM is a **fast and efficient** boosting algorithm ***designed for large datasets***. It uses a leaf-wise growth strategy, unlike the traditional level-wise approach in GBM.

CatBoost:

Handling categorical variables can be challenging, especially when dealing with a large number of them. If a categorical feature has too many unique labels (i.e., high cardinality), applying one-hot encoding can drastically increase the dataset's dimensionality, making it difficult to process and analyze.

CatBoost simplifies this by automatically handling categorical variables without requiring extensive preprocessing, unlike many other machine learning algorithms. This makes it a powerful choice for datasets with numerous categorical features.

Implementing CatBoost in Python

One of the key advantages of CatBoost is that it eliminates the need for one-hot encoding. Simply load the dataset, handle missing values, and we're ready to train the model.

What are the benefits of boosting?

Boosting offers the following major benefits:

Ease of implementation

Boosting has easy-to-understand and easy-to-interpret algorithms that learn from their mistakes. These algorithms don't require any data preprocessing, and they have built-in routines to handle missing data. In addition, most languages have built-in libraries to implement boosting algorithms with many parameters that can fine-tune performance.

Reduction of bias

Bias is the presence of uncertainty or inaccuracy in machine learning results. Boosting algorithms combine multiple weak learners in a sequential method, which iteratively improves observations. This approach helps to reduce high bias that is common in machine learning models.

Computational efficiency

Boosting algorithms prioritize features that increase predictive accuracy during training. They can help to reduce data attributes and handle large datasets efficiently.

What are the challenges of boosting?

The following are common limitations of boosting modes:

Vulnerability to outlier data

Boosting models are vulnerable to outliers or data values that are different from the rest of the dataset. Because each model attempts to correct the faults of its predecessor, outliers can skew results significantly.

Real-time implementation

You might also find it challenging to use boosting for real-time implementation because the algorithm is more complex than other processes. Boosting methods have high adaptability, so you can use a wide variety of model parameters that immediately affect the model's performance.

Conclusion

Boosting is a powerful ensemble learning technique that **reduces bias, enhances model accuracy, and is effective on imbalanced datasets**. Compared to bagging, boosting focuses on learning from mistakes, making it particularly useful for tasks requiring high predictive power.

Among the boosting techniques:

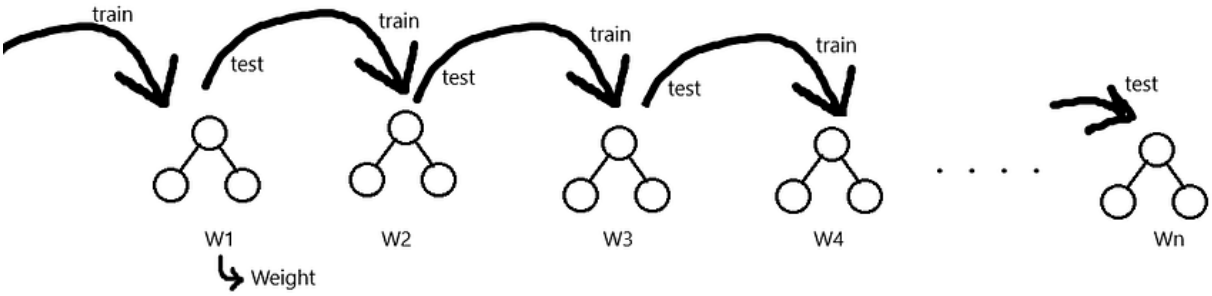
- **AdaBoost** is simple and effective for classification problems.
- **Gradient Boosting** offers flexibility and can be tuned for better performance.
- **XGBoost** is optimized for speed and accuracy with built-in regularization.
- **LightGBM** is ideal for large datasets and faster training.

Choosing the right boosting algorithm depends on your dataset size, required speed, and model complexity. Implementing these techniques can significantly **enhance machine learning performance and prediction accuracy**.

AdaBoost In more information

AdaBoost (adaptive boosting) is a specific boosting algorithm that commonly uses decision trees as weak learners

Press enter or click to view image in full size



We can understand this by following:

Press enter or click to view image in full size

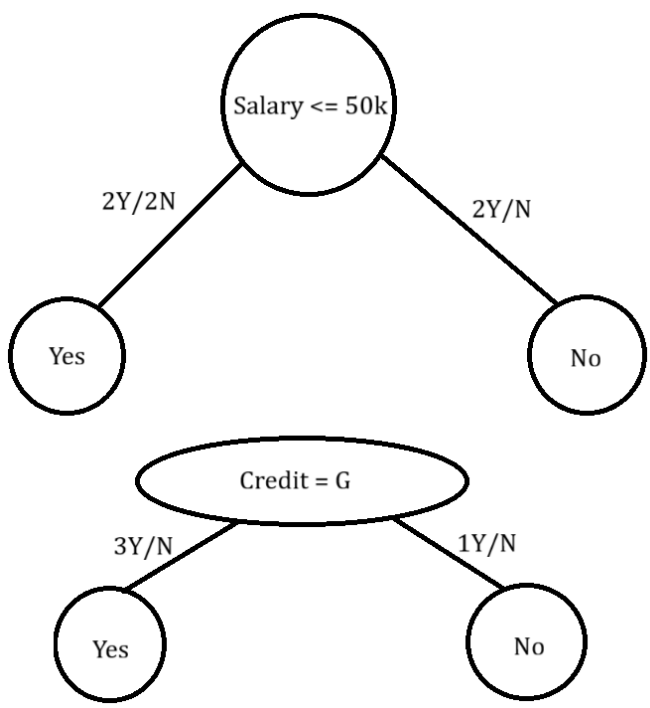
$$F = \alpha_1 M_1 + \alpha_2 M_2 + \alpha_3 M_3 + \alpha_4 M_4 + \alpha_5 M_5 + \ldots\ldots + \alpha_n (M_n)$$

$$\underbrace{M_1, M_2, M_3, m_4, \ldots\ldots, M_n}_{\text{Weak Learners}} \qquad \underbrace{\alpha_1, \alpha_2, \alpha_3, \alpha_4, \ldots\ldots, \alpha_n}_{\text{Weights}}$$

Let's see an example:

Salary	Credit	Approval
<=50k	B	No
<=50k	G	Yes
<=50k	G	Yes
>50k	B	No
>50k	G	Yes
>50k	N	Yes
<=50k	N	No

Step 1: We create a DT stump:



Step 2: Assign Weights

For each row 1/7 weight will be allocated.

Now we have to calculate total error

we can see that our stump has predicted 1 outcome wrong, which is when credit is normal (N) it has declined the approval, so total error is the added weights of “wrong” data points.

Total error (TE) = 1/7

Step 3: Calculate Performance of stump

$$\text{performace of stump} = \frac{1}{2} ln[\frac{1-TE}{TE}]$$

So we get approx. 0.896 as performance of stump

so we will insert the value of performance of our first stump into α1 (alpha_1) = 0.896

Press enter or click to view image in full size

$$F = \alpha_1 M_1 + \alpha_2 M_2 + \alpha_3 M_3 + \alpha_4 M_4 + \alpha_5 M_5 + \ldots\ldots + \alpha_n (M_n)$$

Function for ensemble bethod from AdaBoost, image by author

Step 4: Update the weights for correctly and incorrectly classified datapoints

$$\text{correctly classified datapoints} = \frac{1}{7} \times e^{-\alpha}$$

$$\text{incorrectly classified datapoints} = \frac{1}{7} \times e^{\alpha}$$

	Salary	Credit	Approval	updated_weights
0	<=50K	B	no	0.058321
1	<=50K	G	yes	0.058321
2	<=50K	G	yes	0.058321
3	>50K	B	no	0.058321
4	>50K	G	yes	0.058321
5	>50K	N	yes	0.058321
6	<=50K	N	no	0.349927

Now, normalize the weights and assign bins

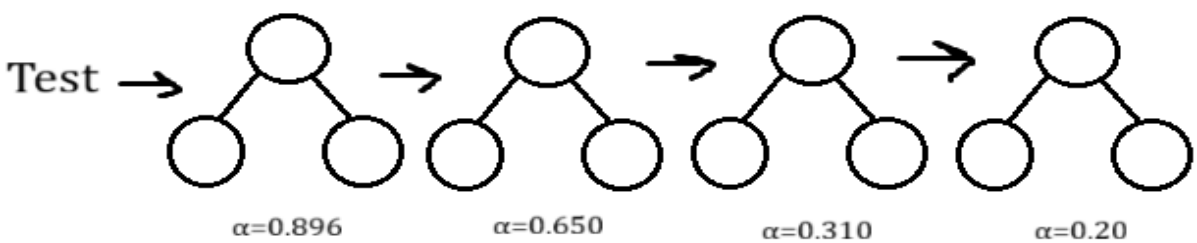
	Salary	Credit	Approval	updated_weights	normalized_weights	Bins
0	<=50K	B	no	0.058321	0.083333	0.0 - 0.08
1	<=50K	G	yes	0.058321	0.083333	0.08 - 0.17
2	<=50K	G	yes	0.058321	0.083333	0.17 - 0.25
3	>50K	B	no	0.058321	0.083333	0.25 - 0.33
4	>50K	G	yes	0.058321	0.083333	0.33 - 0.42
5	>50K	N	yes	0.058321	0.083333	0.42 - 0.5
6	<=50K	N	no	0.349927	0.500000	0.5 - 1.0

Similarly AdaBoost algorithm will assign performance weight for every stump and then add all those together to make a single “function”

Press enter or click to view image in full size

$$F = \alpha_1 M_1 + \alpha_2 M_2 + \alpha_3 M_3 + \alpha_4 M_4 + \alpha_5 M_5 + \ldots\ldots + \alpha_n (M_n)$$

Lets assume the algorithm calculated the performance weight of first four stumps, remember, the selection of stumps is done randomly.



so, by putting in the values for stump(model) weights we

F = 0.896(M1) + 0.650(M2) + 0.310(M3) + 0.20(M4)

Now, assuming we get yes from M1, no from M2, yes from M3 and no from M4, we will finally get YES as a result

This is how AdaBoost works.